Literature Review

for

BrowseBlock

AI browser extension for real-time attack prevention.

Team 16

Prepared by,

Deon George(SJC23CS086)

Amal Kurian Royce (SJC23CS037)

Maria Royce (SJC23CS159)

Melbin K Thomas (SJC23CS164)

St. Joseph's College of Engineering and Technology, Palai

11-10-2026

Literature Review

1 URL / Fake QR Phishing

a) Summary of the Paper: "XGBoost-Based URL Phishing Detection Method With Cross-Dataset Validation" by Milosz Misiek and Tomasz Hyla, proposes a machine learning approach for identifying phishing URLs using the XGBoost algorithm. The method extracts URL features and classifies web addresses as legitimate or malicious. The model is evaluated on three different datasets to assess its accuracy and ability to generalize to unseen data. It achieves 90.7% accuracy and a 91.2% F1-score on the custom dataset, with an average accuracy of 77.84% across three test datasets. The study highlights XGBoost's computational efficiency, its ability to operate without GPU acceleration, and its feature-importance analysis. However, detection performance decreases across different datasets, emphasizing the need for reliable feature selection, continuous model updates, and further validation.

b) Relevance to BrowseBlock: This paper directly supports BrowseBlock's URL / Fake QR Phishing Detection feature by providing an XGBoost-based approach for identifying malicious URLs using extracted URL characteristics. BrowseBlock can adopt this approach by normalizing URLs, extracting relevant features, and using the trained XGBoost model to estimate phishing risk. URLs decoded from QR codes can undergo the same analysis, allowing BrowseBlock to warn users about or block suspicious destinations. The model's computational efficiency makes it suitable for integration with BrowseBlock's browser extension and backend architecture. However, QR-code decoding must be implemented separately, and the model should be tested on diverse, up-to-date datasets to evaluate its generalization and minimize false positives before deployment.

2 HTML / JavaScript Redirects

a) Summary of the Paper: The paper "Hidden Path: Understanding the Intermediary in Malicious Redirections" (2022) by Yuwei Zeng, Zhicheng Liu, Xunxun Chen, and Tianning Zang investigates

how attackers use URL redirections to hide malicious websites and bypass security checks. The authors develop a detection framework that identifies suspicious intermediary domains, uses a dynamic Chrome-based crawler to observe webpage behavior, and reconstructs redirection chains from browser performance logs. The study examines HTTP-based, JavaScript-based, and HTML-based redirects, including delayed redirects, fabricated user clicks, and iframe-based techniques. The findings show that malicious redirections often involve multiple hops and diverse domains, with JavaScript playing a significant role. The paper provides practical methods for identifying concealed redirection paths and understanding how users are redirected to malicious destinations.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's HTML / JavaScript Redirects feature because it explains how attackers use intermediate URLs, multiple redirections, delayed navigation, and JavaScript-based techniques to conceal malicious destinations. BrowseBlock can adopt the paper's redirection-chain monitoring and reconstruction approach to track navigation paths and identify suspicious HTTP, JavaScript, and HTML redirects. Additional methodologies can include Chrome's webNavigation API for monitoring real-time navigation events, URL reputation checking, redirect risk scoring, and declarativeNetRequest rules for blocking malicious destinations. These techniques can help BrowseBlock classify navigation as safe, suspicious, or malicious and warn users or prevent risky navigation where browser capabilities permit. However, the paper's large-scale crawling framework cannot be directly transferred to a lightweight browser extension and must be adapted for real-time operation.

3 Malicious File Downloads

a) Summary of the Paper: The paper *"Hidost: A Static Machine-Learning-Based Detector of Malicious Files"* (2016) by Nedim Šrndić and Pavel Laskov presents a static malware detection system that identifies malicious files by analyzing their logical structure and content without executing them. The authors develop Hidost, which extracts structural features from files, consolidates similar structural paths, selects relevant features, and applies machine learning to classify files as benign or malicious. The system is evaluated on PDF and SWF (Flash) files using datasets containing approximately 440,000 PDF files and 40,000 SWF files. The results demonstrate strong detection performance, with Hidost detecting more

malicious PDF files than the antivirus engines evaluated through VirusTotal and ranking among the best for SWF malware. The study highlights the potential of structural feature extraction and static machine learning for efficient malware detection while discussing limitations such as dynamically loaded threats and the need to update detection models as malware patterns evolve.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Malicious File Download Detection** feature because it demonstrates how static analysis and machine learning can identify malicious content within downloaded files without executing them. BrowseBlock can apply similar principles by extracting file-specific features, such as PDF JavaScript actions, embedded files, document structures, executable file properties, and suspicious archive contents, before classifying files using its proposed XGBoost model and deterministic security rules. The paper's structural path consolidation technique also provides a useful research direction for reducing feature complexity and improving the representation of structurally similar files. These approaches can help BrowseBlock assess downloaded files and allow, warn about, or block potentially malicious content where browser capabilities permit. However, Hidost was evaluated on PDF and SWF formats, so its results cannot be directly generalized to BrowseBlock's other target formats, including EXE, ZIP, DOC, and DOCX. Each format requires suitable feature extraction, training data, and independent evaluation.

4 E-mail Phishing

a) Summary of the Paper: The paper *"Classification of Phishing Email Using Random Forest Machine Learning Technique"* (2014) by Andronicus A. Akinyelu and Aderemi O. Adewumi investigates the use of the Random Forest machine-learning algorithm to classify emails as phishing or legitimate. The authors extract 15 phishing-related features from email messages, including IP-based URLs, mismatches between link text and actual destinations, suspicious keywords, JavaScript presence, the number of links, and differences between the sender's domain and domains referenced in the email. The extracted features are represented numerically and used to train and evaluate a Random Forest classifier through 10-fold cross-validation. Experiments conducted using a dataset of 2,000 emails report a classification accuracy of 99.7%, a false-positive rate of 0.06%, and a false-negative rate of 2.50% for the largest dataset configuration.

The study demonstrates the potential of feature-based machine learning for phishing email detection while emphasizing the importance of selecting relevant phishing-specific features to improve classification performance.

b) Relevance to BrowseBlock: This paper is directly relevant to BrowseBlock's **Email Phishing Detection** feature because it demonstrates how a Random Forest classifier can identify phishing emails using measurable characteristics of email content and embedded links. BrowseBlock can adopt similar features, including suspicious keywords, URL counts, IP-based URLs, mismatches between displayed link text and actual destinations, and sender-domain inconsistencies, to build its initial email-screening model. The resulting Random Forest prediction can be combined with BrowseBlock's URL phishing detection model, email authentication signals such as SPF, DKIM and DMARC, and RAG/LLM-based semantic analysis to produce a more comprehensive risk assessment. Based on the final verdict, BrowseBlock can display a warning, flag suspicious messages, and block malicious links or associated navigation where browser capabilities permit. However, the paper was published in 2014 and evaluated a relatively small dataset, so its reported accuracy should not be assumed to represent performance on modern phishing attacks. BrowseBlock must train and evaluate its model on representative, up-to-date datasets and measure false positives and false negatives before deployment.

5 Cryptojacking

a) Summary of the Paper: The paper *"Analyzing In-Browser Cryptojacking"* (2024) by Muhammad Saad and David Mohaisen investigates cryptojacking attacks in which malicious JavaScript secretly uses website visitors' computing resources to mine cryptocurrencies without their consent. The authors analyze more than 5,700 websites associated with cryptojacking and examine the characteristics of malicious mining scripts through static and dynamic analysis. For static analysis, they extract JavaScript code features, including cyclomatic complexity, Halstead complexity measures, and source-code characteristics, and apply machine-learning algorithms such as Logistic Regression, Linear Discriminant Analysis, Support Vector Machines, k-Nearest Neighbors, and Random Forest to distinguish cryptojacking scripts from benign and other malicious JavaScript. Their experiments report strong classification performance,

with several models achieving 100% precision, recall, and F1-score on the evaluated datasets. Dynamic analysis investigates the impact of cryptojacking on CPU consumption, battery drainage, and WebSocket communication with remote mining servers. The findings show that cryptojacking can significantly consume system resources and drain device batteries. The paper also evaluates the economic feasibility of cryptojacking and proposes countermeasures based on its findings.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Cryptojacking Detection** feature because it demonstrates how static JavaScript analysis and machine learning can identify scripts associated with unauthorized cryptocurrency mining. BrowseBlock can adopt similar approaches by extracting JavaScript code-complexity features, examining suspicious script behavior, and using a trained classification model to distinguish potentially malicious mining scripts from benign scripts. The paper's dynamic analysis findings also provide useful indicators for BrowseBlock, including unusually high CPU consumption and suspicious WebSocket communication patterns. These indicators can complement static analysis to improve detection and help BrowseBlock classify websites as safe, suspicious, or potentially malicious. Based on the risk assessment, BrowseBlock can warn users and block known malicious script requests or other supported network activity using browser-extension mechanisms. However, the paper's reported classification results should not be assumed to guarantee the same accuracy in BrowseBlock, particularly because the first experiment used a limited number of distinct cryptojacking scripts. The proposed extension must be evaluated against current scripts, obfuscation techniques, and legitimate resource-intensive JavaScript to reduce false positives and detection gaps.

6 XSS scripting

a) Summary of the Paper: The paper *"Detecting DOM-Sourced Cross-Site Scripting in Browser Extensions"* (2017) by Jinkun Pan and Xiaoguang Mao proposes a hybrid analysis framework to detect DOM-sourced Cross-Site Scripting (XSS) vulnerabilities in browser extensions, particularly Greasemonkey user scripts. The authors explain how attackers can exploit untrusted webpage DOM content to inject malicious JavaScript into vulnerable extension scripts, potentially abusing their privileged capabilities to access cross-origin resources. The proposed framework combines lightweight static analysis with dy-

namic symbolic execution. Static analysis uses a text filter and an Abstract Syntax Tree (AST) parser to identify suspicious script metadata, overly generic inclusion rules, privileged API usage, potential DOM sources, and dangerous execution sinks such as `eval()`. Scripts that remain suspicious undergo dynamic symbolic execution supported by a Shadow DOM component, which maintains the document structure and helps generate proof-of-concept exploits to verify vulnerabilities. The authors evaluated their approach on 154,065 Greasemonkey user scripts and identified 58 previously unknown DOM-sourced XSS vulnerabilities. The study demonstrates the effectiveness of combining static and dynamic analysis, while acknowledging that dynamic analysis is computationally expensive and incomplete support for DOM operations may cause missed vulnerabilities.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Cross-Site Scripting (XSS) Detection and Prevention** feature because it demonstrates how static code analysis and DOM-related data-flow analysis can identify potentially dangerous JavaScript execution paths. BrowseBlock can adopt lightweight techniques inspired by the paper, such as examining JavaScript syntax using an AST parser, identifying suspicious DOM inputs, and checking for dangerous execution sinks such as `eval()`. These indicators can be combined with selective DOM monitoring and browser-supported request-blocking mechanisms to assess suspicious activity and prevent access to known malicious scripts where browser capabilities permit. The paper also supports BrowseBlock's approach of filtering potential threats locally before performing more expensive analysis. However, the original framework targets vulnerabilities in Greasemonkey user scripts and relies on dynamic symbolic execution and a Shadow DOM, which are computationally expensive for continuous browser-extension monitoring. BrowseBlock must therefore adapt these techniques into a lightweight, event-driven detection pipeline and evaluate its effectiveness against reflected, stored, and DOM-based XSS attacks. Furthermore, detecting suspicious DOM changes alone cannot guarantee prevention of JavaScript that has already executed; prevention requires appropriate browser-level controls and careful risk-based enforcement.

7 Clickjacking

a) Summary of the Paper: The paper *"Implementation of Extension for Browser to Detect Vulnerable Elements on Web Pages and Avoid Clickjacking"* (2016) by Prof. Dipti Pawade, Abhilasha Lahigude, Divya Reja, and Prof. Era Johri presents ClickProtect, a JavaScript-based Google Chrome extension designed to detect and mitigate clickjacking attacks. The paper focuses on two attack techniques: invisible or transparent iframe overlays and fake cursors that mislead users into clicking unintended elements. The proposed extension consists of two main components: a revealing unit that scans webpage source code to identify clickable elements, checks their visibility properties, and detects cursor customization; and an extenuation unit that warns users about potential attacks and takes preventive actions. When an invisible iframe or clickable element is detected, ClickProtect increases its visibility and displays a warning message. When a fake cursor is detected, the extension restores the original cursor and notifies the user. The experimental results demonstrate how the extension identifies invisible elements and suspicious cursor modifications, helping users recognize potentially deceptive interactions. The authors also identify the need to extend the solution to additional clickjacking variants and tapjacking attacks.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Clickjacking Detection and Prevention** feature because it demonstrates a browser-extension-based approach to identifying deceptive webpage elements that trick users into performing unintended actions. BrowseBlock can adopt similar techniques by inspecting clickable elements, detecting suspiciously transparent or hidden iframes, examining CSS visibility and stacking properties, and identifying attempts to hide or replace the original cursor. When suspicious behavior is detected, BrowseBlock can display a warning, highlight potentially deceptive elements, and help users avoid unsafe interactions. The paper's two-component architecture also provides a useful design reference for separating threat detection from user notification and preventive actions. However, checking iframe visibility and cursor customization alone cannot reliably detect every clickjacking attack, and legitimate websites may also use transparent elements or custom cursors. BrowseBlock should therefore correlate multiple indicators, minimize false positives, and evaluate its detection methods against different clickjacking techniques. Where possible, browser-supported security controls and website-level protections such as Content Security Policy frame-ancestors directives can complement

the extension's detection mechanisms.

8 Browser Extension Abuse

a) Summary of the Paper: The paper *"CPDetector: Automated Detection of Excessive Permissions in Chrome Extensions"* by Xiaoyu Cheng, Zhi Wang, Wanpeng Li, Wei Zeng, and Xin Yang proposes a framework for identifying Chrome browser extensions that request more permissions than they actually require. The authors develop CPDetector, which extracts declared permissions from the `manifest.json` file and compares them with Chrome extension API usage identified through static JavaScript analysis. The framework also examines host permissions to identify extensions requesting unnecessarily broad access to websites. Using a dataset of 199,058 Chrome Web Store extensions collected as of May 2025, the researchers found that 49.3% (98,072 extensions) exhibited excessive permission authorization. Their analysis also identified 33,998 extensions with excessive host permissions among 129,953 extensions examined for this aspect. To help users understand these risks, the authors developed PermissionGuard, a Chrome extension that identifies over-privileged extensions and displays permission risk warnings. The study highlights how excessive permissions can expose browsing history, cookies, and website data to unauthorized access, emphasizing the importance of the principle of least privilege in browser extension security.

b) Relevance to BrowseBlock: This paper is relevant to BrowseBlock's **Browser Extension Abuse Detection** feature because it demonstrates how excessive permissions and broad website access can indicate potential security and privacy risks associated with browser extensions. BrowseBlock can adopt similar techniques by analyzing an extension's `manifest.json`, identifying requested API and host permissions, and comparing them with the APIs and website domains referenced in its source code. Permissions such as `cookies`, `history`, and broad host access like `<all_urls>` can be highlighted for further risk assessment when they appear unnecessary for the extension's stated functionality. BrowseBlock can also incorporate permission-risk scoring, extension metadata checks, and user-facing warnings to help users identify potentially over-privileged extensions. However, the paper primarily detects excessive permissions rather than proving that an extension is malicious. BrowseBlock should therefore treat permission mismatches as risk indicators, not definitive evidence of abuse, and combine them with additional evidence

such as suspicious script injection, unauthorized data collection, remote code behavior, and changes in extension versions. The paper also notes limitations in static analysis, so dynamic behavior and context-dependent API usage may require additional investigation.

9 Browser Notification Scams

a) Summary of the Paper: The paper *"When Push Comes to Ads: Measuring the Rise of (Malicious) Push Advertising"* by Karthika Subramani, Xingzi Yuan, Omid Setayeshfar, Phani Vadrevu, Kyu Hyung Lee, and Roberto Perdisci (2020) investigates the misuse of web push notifications to distribute malicious advertisements and scams. The authors introduce **PushAdMiner**, a system designed to collect web push notifications, identify advertising campaigns, and discover potentially malicious campaigns. The system uses an instrumented Chromium browser to monitor notification permissions, Service Worker activity, notification content, network requests, and the URLs users reach after clicking notifications. It then applies text similarity, URL-path comparison, clustering, and threat-intelligence services such as Google Safe Browsing and VirusTotal to identify suspicious campaigns. The researchers collected 21,541 web push notification messages and identified 572 advertising campaigns containing 5,143 advertisements. They reported that 51% of the collected push advertisements were malicious. The study also demonstrated that conventional ad blockers and URL blocklists could miss malicious notification campaigns, highlighting the need for improved detection techniques.

b) Relevance to BrowseBlock: This paper is directly relevant to BrowseBlock's **Browser Notification Scam Detection** feature. It demonstrates how web push notifications can be abused to deliver fake security alerts, fraudulent offers, phishing links, and technical-support scams, even when users are no longer visiting the website that originally requested notification permission. BrowseBlock can apply ideas from the paper by examining notification titles, message bodies, destination URLs, domain information, and redirect chains to identify suspicious notifications. URL reputation checks can be combined with local detection rules and the existing URL phishing detection model to assess the linked destination. BrowseBlock can also identify warning patterns such as fake account-security alerts, misleading rewards, urgent payment warnings, and notifications that redirect users to unrelated domains. When a notification

is suspicious, the extension can warn the user and assess its destination before navigation, where browser APIs permit. The paper also highlights the limitations of relying exclusively on URL blocklists, supporting BrowseBlock's approach of combining multiple detection signals. However, PushAdMiner primarily focuses on collecting, measuring, and analyzing malicious advertising campaigns rather than implementing a lightweight, real-time browser notification protection extension. Therefore, BrowseBlock can use the paper as a research foundation, while developing and evaluating its own real-time detection and prevention mechanism. Browser notification interception and blocking capabilities must also be verified against the specific browser APIs and permissions available to the extension.

10 Credential Harvesting

a) Summary of the Paper: The article *"Web Scraping with Playwright and Python"* by Scrapfly explains how to use Playwright, a browser automation framework, with Python to interact with modern websites and extract information from dynamically rendered web pages. Unlike basic HTTP scraping tools, Playwright can control a real browser engine, execute JavaScript, interact with page elements, and wait for content to appear. It supports browser navigation, element selection through locators, interaction with forms and buttons, and extraction of page content. These capabilities make it suitable for websites that rely on JavaScript to generate or modify their content. Playwright can also be used to automate repeatable browser interactions and test website behavior. Its official Python documentation describes navigation handling, browser contexts, locators, and browser interactions, providing a foundation for building automated web testing and analysis workflows.

b) Relevance to BrowseBlock: This tool is relevant to BrowseBlock's **Credential Harvesting Detection** feature because it can automate the inspection of websites that may imitate legitimate login pages to steal usernames, passwords, and other sensitive information. BrowseBlock can use Playwright in its testing and analysis environment to load suspicious websites, inspect dynamically generated login forms, identify password fields and credential-submission forms, and observe form actions, page URLs, and navigation behavior. Playwright's locator APIs can identify elements using labels, placeholders, roles, and CSS selectors, while its navigation and network-event capabilities can help investigate redirects and

requests associated with form submission. These observations can support the development and validation of detection rules that compare the displayed website identity with the actual domain receiving submitted credentials. Combined with BrowseBlock's URL phishing classifier, domain reputation checks, and risk-scoring system, these features can help identify fake login pages and evaluate whether suspicious credential-harvesting attempts should trigger a warning or blocking action. However, Playwright is best suited to controlled analysis, automated testing, and dataset generation rather than continuous monitoring of every user's browsing session. The production browser extension should perform lightweight, event-driven checks using browser-extension APIs, while Playwright can test those checks against legitimate login pages and simulated phishing pages. Testing must use synthetic credentials and controlled environments; Playwright alone cannot determine whether a login page is malicious or guarantee that credentials will be intercepted.